Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Actividad Autónoma — Sesión 10

Pruebas de seguridad y gestión de contraseñas en PharmaBackend

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	10 · 15 de octubre de 2026
Modalidad	Individual, sin acompañamiento docente	Plazo	Hasta el 21 de octubre de 2026, 23:59
Entrega	Aula virtual + repositorio Git	Puntaje	20 puntos (evaluación de sesión)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. De dónde partimos

En la actividad práctica (Reto 01) PharmaBackend aprendió a preguntar quién llama y qué puede hacer: las contraseñas se guardan con BCrypt, el login entrega un JWT y cada endpoint exige su permiso. Pero una seguridad que no se prueba es solo una suposición. ¿Qué pasa si alguien altera el token, si lo envía sin el prefijo `Bearer`, si el administrador desactiva a un vendedor que ya tenía sesión o si un almacenero intenta registrar una venta?

Además, la administradora de la botica pide tres cosas: que cada trabajador pueda **cambiar su propia contraseña**, que ella pueda **cambiar los roles** de un usuario cuando cambia de puesto y que pueda **desactivar** a quien deja de trabajar, sin borrar su historial.

Reto 02

Prueba sistemáticamente la autenticación, los tokens y los permisos de PharmaBackend con una matriz de casos medidos, e implementa el cambio de contraseña del usuario autenticado, la asignación de roles y la activación o desactivación de usuarios por el administrador.

2. Actividad autónoma

Esta actividad extiende el Reto 01 sobre el mismo repositorio. La primera parte es de **verificación**: cada caso tiene una precondición, unos datos, un resultado esperado y el resultado obtenido. La segunda parte es de

construcción: dos endpoints nuevos que siguen la arquitectura que ya conoces. Si te bloqueas, revisa la sección 4, «Dudas frecuentes».

Campo	Contenido
Título	Matriz de pruebas de seguridad, cambio de contraseña, asignación de roles y desactivación de usuarios
Objetivo	Verifica la autenticación JWT y la autorización por permisos de PharmaBackend contrastando resultados esperados y obtenidos, e implementa el cambio de contraseña con BCrypt, la asignación de roles (relación muchos a muchos) y la desactivación de usuarios respetando las reglas de seguridad.
Tiempo estimado	4 a 5 horas
Materiales	Guía práctica de la sesión 10, PharmaBackend (rama `feature/seguridad-jwt`), Swagger UI y Postman.

Preparación

- Crea la rama `feature/seguridad-pruebas-<apellido>` desde tu rama de la práctica.
- Crea en Postman la colección `PharmaSoft – S10 Seguridad` con las variables `baseUrl = http://localhost:8080/api/v1`, `tokenAdmin`, `tokenVendedor` y `tokenAlmacen`.
- En la pestaña **Tests** de cada login guarda el token automáticamente: `pm.collectionVariables.set("tokenVendedor", pm.response.json().token);`.

Parte A. Matriz de pruebas de seguridad (8 puntos)

Construye la matriz con las columnas **ID · Grupo · Usuario / token · Petición · Resultado esperado (código y mensaje) · Resultado obtenido · Estado (Pasa / Falla) · Evidencia**. La matriz debe tener **al menos 20 casos**: los 16 casos base y **al menos 4 propios**, cada uno de un grupo distinto.

ID	Grupo	Caso	Resultado esperado
A-01	Autenticación	Login de admin, vendedor, almacen y supervisor.	200; cada respuesta trae sus roles y sus permisos; la supervisora, la unión de VENDEDOR y ALMACENERO.
A-02	Autenticación	Login con usuario inexistente y luego con contraseña incorrecta.	401 con el mismo mensaje en ambos casos.
A-03	Autenticación	Login de `inactivo` con su contraseña correcta.	401; el mensaje no revela que la cuenta existe.
A-04	Autenticación	Login escribiendo el usuario en mayúsculas (`VENDEDOR`).	Decide la regla esperada y compárala con lo que hace la API.
T-01	Token	Petición protegida con `Authorization: <token>` sin la palabra Bearer.	401 «Debe iniciar sesión…».
T-02	Token	Token con un carácter cambiado en el payload.	401 «El token no es válido».
T-03	Token	Token emitido con otro secreto (cambia `app.jwt.secret`, reinicia y usa un token anterior).	401 «El token no es válido».
T-04	Token	Token vencido (`expiracion-minutos: 1`).	401 «El token ha expirado…».

ID	Grupo	Caso	Resultado esperado
T-05	Token	Decodificar el payload y comprobar `sub`, `authorities`, `iat` y `exp`.	Datos correctos; ningún dato sensible en el payload.
P-01	Permisos	VENDEDOR: `POST`, `PUT` y `DELETE` de categorías.	403 en los tres.
P-02	Permisos	ALMACENERO: `GET /clientes` y `POST /ventas`.	403 en ambos.
P-03	Permisos	VENDEDOR y ALMACENERO: `GET /reportes/productos-mas-vendidos`.	403 (rechazo por `@PreAuthorize`).
P-04	Permisos	ADMIN: una petición de cada módulo.	2xx en todas.
U-01	Usuarios	ADMIN crea un usuario con contraseña `abcdefgh` y luego con `rolIds: []`.	400 en ambos (número obligatorio en la contraseña; al menos un rol).
U-02	Usuarios	ADMIN crea un usuario para un empleado que ya tiene cuenta y luego para María Huamán; revisar `usuarios` y `usuario_roles` en SQL Developer.	409 (relación 1 a 1) y luego 201; la respuesta no trae contraseña, `password` guarda `$2a$10$…` y hay una fila nueva en `usuario_roles`.
C-01	CORS	Desde la SPA (`localhost:4200`), observar en la pestaña Red la petición `OPTIONS` previa a un `GET` con token.	La verificación responde 200 y la petición real 200 o 401, nunca un error de CORS.

Cómo registrar la evidencia

Cada caso necesita una captura de Postman o Swagger donde se vean la petición (método, URL y encabezado Authorization), el código HTTP y el cuerpo de la respuesta. Nombra las capturas con el ID: `T-02_postman.png`. Nunca publiques un token real de producción.

Parte B. Contraseña, roles y estado de los usuarios (7 puntos)

B1. Cambio de contraseña del usuario autenticado (3 puntos). Cualquier usuario autenticado puede cambiar **su propia** contraseña:

```
PUT /api/v1/auth/password          (requiere token, cualquier rol)
{
  "passwordActual": "Vendedor2026*",
  "passwordNueva":  "NuevaClave2026"
}
→ 204 No Content
```

- DTO `CambioPasswordRequestDTO` con las mismas reglas de contraseña de `UsuarioRequestDTO` para `passwordNueva` (8 a 72 caracteres, al menos una letra y un número).
- El usuario se obtiene del token (`Authentication`), **nunca** de un id enviado en el cuerpo.
- Si `passwordActual` no coincide con el hash (`passwordEncoder.matches`) → **409** «La contraseña actual no es correcta».
- Si la nueva es igual a la actual → **409** «La nueva contraseña debe ser distinta de la actual».
- Se guarda `passwordEncoder.encode(nueva)` y se registra un log INFO con el usuario, **sin** contraseñas.

B2. Asignar roles a un usuario (2 puntos). Solo el ADMIN. Reemplaza el conjunto de roles del usuario (relación muchos a muchos):

```
PUT /api/v1/usuarios/{id}/roles
{ "rolIds": [2, 3] }
→ 200 con UsuarioResponseDTO (roles y permisos actualizados)
```

- DTO `AsignarRolesRequestDTO` con `@NotEmpty Set<@NotNull Long> rolIds` → **400** si viene vacío.
- Usuario inexistente o algún rol inexistente → **404**. Un ADMIN no puede quitarse a sí mismo el rol ADMIN → **409**.
- Basta con `usuario.setRoles(new HashSet<>(roles))`: JPA actualiza las filas de `usuario_roles`.
- Comprueba que, con el **mismo** token, el usuario obtiene el nuevo permiso en su siguiente petición, y explica por qué en el informe (pista: revisa qué hace `JwtAuthenticationFilter` con el username).

B3. Activar o desactivar usuarios (2 puntos). Solo el ADMIN (permiso `USUARIOS_ADMINISTRAR`):

```
PATCH /api/v1/usuarios/{id}/estado
{ "estado": false }
→ 200 con UsuarioResponseDTO
```

- Usuario inexistente → **404**. Un ADMIN no puede desactivarse a sí mismo → **409**.
- Comprueba que el token que el usuario desactivado ya tenía **deja de funcionar** en su siguiente petición (401 «El usuario está deshabilitado») y que su login responde 401.
- Agrega los casos B-01 a B-08 de las partes B1, B2 y B3 a tu matriz (no cuentan dentro de los 4 propios).

Parte C. Informe de seguridad (3 puntos)

Redacta un informe breve (2 a 3 páginas) con:

- **C1. Resumen:** casos ejecutados, cuántos pasan y cuántos fallan por grupo.
- **C2. Fichas de hallazgos:** una por caso que falla, con descripción, severidad, evidencia, causa probable (clase y método) y corrección propuesta.
- **C3. Preguntas de análisis**, respondidas en un párrafo cada una:

1. ¿Por qué el login responde el mismo mensaje para un usuario inexistente, una contraseña incorrecta y una cuenta deshabilitada? ¿Qué información le daría a un atacante un mensaje distinto?
2. Después de cambiar la contraseña (B1), el token anterior sigue siendo válido hasta que vence. ¿Por qué ocurre en una API sin estado? Propón una forma de invalidarlo.
3. Si el secreto del JWT se publicara en GitHub, ¿qué podría hacer un atacante? ¿Qué harías de inmediato?
4. ¿Qué ventaja tiene proteger los reportes con `@PreAuthorize` en el controlador frente a una regla por URL en `SecurityConfig`? ¿Y qué desventaja?

Parte D. Versionamiento y entrega (2 puntos)

- Al menos tres commits en días distintos en `feature/seguridad-pruebas-<apellido>`; por ejemplo: `test(seguridad): matriz de pruebas de autenticación y permisos`, `feat(auth): cambio de contraseña del usuario autenticado`, `feat(usuarios): asignar roles y activar o desactivar usuarios`.

- Guarda en el repositorio `docs/matriz-seguridad-s10.xlsx` (o `.md`), `docs/informe-s10.pdf` y la colección `postman/PharmaSoft-S10-Seguridad.json` **sin tokens reales** en sus variables.
- Abre una solicitud de incorporación hacia `master` y sube el informe al aula virtual como `Apellido_LP2_S10_Autonoma.pdf`.

Producto esperado

- Matriz con al menos 20 casos ejecutados (más B-01 a B-08), con código y mensaje esperados, resultado obtenido, estado y evidencia.
- Endpoints `PUT /api/v1/auth/password`, `PUT /api/v1/usuarios/{id}/roles` y `PATCH /api/v1/usuarios/{id}/estado` funcionando según sus reglas.
- Informe de seguridad con las secciones C1 a C3.
- Rama con al menos tres commits, colección de Postman y solicitud de incorporación hacia `master`.

Lista de cotejo

N.º	Criterio observable	Pts.	Sí	No
1	La matriz tiene al menos 20 casos, con los 16 base y 4 propios de grupos distintos.	2		
2	Cada caso indica código HTTP y mensaje esperados y obtenidos, y su estado.	2		
3	Los casos de token (alterado, de otra clave, vencido, sin Bearer) están ejecutados con evidencia.	2		
4	Los casos de permisos cubren los tres roles y distinguen 401 de 403.	1		
5	Cada caso tiene su evidencia nombrada con el ID y la colección no guarda tokens reales.	1		
6	El cambio de contraseña toma el usuario del token, valida la actual con BCrypt y guarda el nuevo hash.	1.5		
7	El cambio de contraseña responde 204, 400 y 409 según el caso, y no registra contraseñas en el log.	1.5		
8	La asignación de roles reemplaza las filas de `usuario_roles`, responde 400, 404 y 409 según el caso y el nuevo permiso aplica en la siguiente petición.	2		
9	La desactivación es solo para ADMIN, responde 404 y 409 según el caso y el token del desactivado deja de servir.	2		
10	Las fichas de hallazgos indican causa probable y corrección propuesta.	1.5		
11	Las cuatro preguntas de análisis se responden con argumentos y referencias al código.	1.5		
12	La rama tiene al menos tres commits en días distintos, los documentos en `docs/` y la solicitud de incorporación.	2		
	Total	**20**		

3. Criterios de entrega

- Plazo: **21 de octubre de 2026, 23:59**. En la sesión 11 (22 de octubre) la SPA hará login, guardará el token y lo enviará con un interceptor: tu backend seguro será la base.
- Las entregas tardías se califican sobre el 70 % del puntaje, hasta 24 horas después del plazo.
- Una matriz cuyos resultados no coincidan con las evidencias, o un repositorio sin commits propios, se califica con cero.
- Puedes usar asistentes de inteligencia artificial para consultar dudas, pero en clase el docente podrá pedirte que repitas cualquier caso o expliques cualquier línea de tus endpoints.

4. Dudas frecuentes

Pregunta	Respuesta
¿Cómo obtengo el usuario autenticado en el controlador?	Agrega un parámetro `Authentication authentication` al método y usa `authentication.getName()`, como en `GET /auth/me`.
¿Qué excepción uso para «contraseña actual incorrecta»?	`ReglaNegocioException`, que `GlobalExceptionHandler` ya convierte en 409.
En T-03 cambié el secreto y ahora ningún login funciona.	El login sí funciona: lo que falla son los tokens anteriores, que es justo lo que pruebas. Haz login otra vez y restaura el secreto al terminar.
¿Cómo sé si un ADMIN intenta desactivarse a sí mismo?	Compara el username del token (`authentication.getName()`) con el del usuario que se va a desactivar.
En A-04 el login con mayúsculas funcionó. ¿Está mal?	No necesariamente: el repositorio busca sin distinguir mayúsculas. Decide si es la regla correcta y justifícalo en el informe.
¿Cómo reemplazo los roles de un usuario?	Busca los roles con `rolRepository.findAllById(...)`, verifica que lleguen todos y asigna `usuario.setRoles(new HashSet<>(roles))` dentro de un método `@Transactional`.
¿Debo construir una pantalla en Angular?	No. Esta actividad es solo de backend; el login en la SPA es el tema de la sesión 11.

Cuando termines, la pregunta con la que empezó la sesión tendrá una respuesta completa. En el Reto 01 protegiste PharmaBackend: contraseñas con BCrypt, un login que entrega un JWT firmado y permisos que cada rol trae consigo. En el Reto 02 demostraste, con casos medidos, que un token alterado, ajeno o vencido no abre ninguna puerta, que cada rol solo llega a lo que le corresponde y que el administrador puede retirar el acceso a quien ya no debe tenerlo. Esa es la diferencia entre una API que funciona y una API en la que la botica puede confiar.

5. Referencias

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. https://www.rfc-editor.org/rfc/rfc7519

OWASP Foundation. (s. f.-a). *Authentication cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Authentication*Cheat*Sheet.html

OWASP Foundation. (s. f.-b). *JSON Web Token for Java cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/JSON*Web*Token*for*Java*Cheat*Sheet.html

OWASP Foundation. (2023). *Web security testing guide* (versión 4.2). https://owasp.org/www-project-web-security-testing-guide/

Postman. (s. f.). *Authorization types supported by Postman*. https://learning.postman.com/docs/sending-requests/authorization/authorization-types/